Lab 1: Git Workflow & Project Setup

I. Learning Objectives

After completing this lab, you will be able to:

- Apply a Git Feature Branch Workflow: **branch ⇨ commit ⇨ PR ⇨ review ⇨ merge**
- Set up group repository for mini-project with proper collaborator access

II. Prerequisites

- A computer with `git` installed. (Check with `git --version`).
- A GitHub account.
- A code editor (e.g., VS Code).

III. Part A: Individual Warm-up (The Fork & PR Workflow)

In this part, you will practice the full workflow on a sample repository. This simulates working on an open-source project or a project where you don't have direct write access. It's a fundamental skill.

Step A.1: Fork and Clone the Sample Repository

1. Lecturer will provide a link to the sample GitHub repository (e.g., `https://github.com/LecturerUsername/sample-repo`).
2. Go to this link and click the "**Fork**" button in the top-right corner. This creates a personal copy (a "*fork*") of the repo under your own GitHub account.
3. On your GitHub account, go to the repo you just forked (e.g., `https://github.com/YourUsername/sample-repo`).

4. Click the green "**Code**" button and copy the HTTPS URL.
5. Open your Terminal (or Git Bash), `cd` to your projects folder, and run `git clone`. This downloads your fork to your local machine.

```
# Fork & clone the sample repo
$ git clone https://github.com/YourUsername/sample-repo.git

$ cd sample-repo
```

Step A.2: Configure "Upstream"

Your local repo currently only knows about `origin`, which is your fork on GitHub. We need to tell it where the **original** project is, which we call `upstream`. This allows you to pull updates from the lecturer's project later.

```
# Add a new remote named "upstream" pointing to the lecturer's original repo
$ git remote add upstream https://github.com/LecturerUsername/sample-repo.git

# Verify your remotes. You should see two:
# origin https://github.com/YourUsername/sample-repo.git (for pushing)
# upstream https://github.com/LecturerUsername/sample-repo.git (for pulling)
$ git remote -v
```

Step A.3: Create a Feature Branch

We **never** commit directly to the `main` branch. This is the most important rule. Always create a new, descriptive branch for each new task.

```
# 1. First, make sure your local 'main' is up-to-date with the *original* project
```

```
git checkout main
git pull upstream main

# 2. Now, create your new branch based on this up-to-date 'main'
# (git checkout -b is a shortcut for 'git branch ...'
   + 'git checkout ...')
git checkout -b feature/add-your-name
```

Branch Naming: Use prefixes like `feature/`, `fix/`, or `docs/`.
Good: `feature/add-user-login-form`, `fix/navbar-bug`
Bad: `my_branch`, `update`, `test`

Step A.4: Make Your Change

Task: Open the `CONTRIBUTORS.md` file in VS Code and add your name and GitHub username to the list.

Step A.5: Commit Your Change

After saving the file, return to your Terminal and "*commit*" your changes.
A commit is a "*snapshot*" of your work.

```
# 1. Check the status; you should see CONTRIBUTORS.md as modified
git status

# 2. Add the file to the "staging area"
(files to be included in the commit)
git add CONTRIBUTORS.md

# 3. Commit the change with a clear and descriptive message
git commit -m "docs: Add [Your Name] to contributors"
```

Best Practice: Conventional Commits

Your commit message is the most important part of your project's history. We use "*Conventional Commits*" to keep it clean. The format is `type: description`.

- `feat:` A new feature (e.g., `feat: Add user login form`)
- `fix:` A bug fix (e.g., `fix: Correct validation for email field`)
- `docs:` Changes to documentation (e.g., `docs: Update README.md`)
- `style:` Code formatting, missing semi-colons, etc.
- `refactor:` Rewriting code without changing its behavior.
- `ci:` Changes to CI/CD files (e.g., `.github/workflows/ci.yml`)

Step A.6: Push Your Branch to GitHub

Your commit is still only on your local machine. You need to push it to your `origin` (your fork).

```
# Push the 'feature/add-your-name' branch to the remote named 'origin'
$ git push origin feature/add-your-name
```

Step A.7: Open a Pull Request (PR)

1. Go to your forked repo on GitHub (`https://github.com/YourUsername/sample-repo`).
2. You should see a yellow bar: "*feature/add-your-name had recent pushes*". Click the **"Compare & pull request"** button.
3. This is the most important step. Make sure you are requesting to merge:
   - **base repository:** `LecturerUsername/sample-repo` (The original)
   - **base:** `main`

   into

   - **head repository:** `YourUsername/sample-repo` (Your fork)
   - **compare:** `feature/add-your-name`

4. **Write a good PR description.**
    - **Title:** Same as your commit

    (e.g., `docs: Add [Your Name] to contributors`).

    - **Body:** What did you do? Why? How did you test it?

    (e.g., "*This PR adds my name to the CONTRIBUTORS.md file as part of Lab 1.*").

5. Add your Lecturer (and 1-2 classmates) to the "**Reviewers**" list on the right.
6. Click "**Create pull request**".

Step A.8: Practice Code Review

A PR is where collaboration happens. Go find a PR from one of your classmates.

Task 2: Find a classmate's Pull Request.

1. Read their description.
2. Go to the "*Files changed*" tab.
3. Click the `+` icon on a line of code they changed.
4. Leave a constructive comment

   (e.g., "Looks good!" or "Suggestion: Maybe we should add our student ID too?").

5. Click the "**Review changes**" button, select "**Approve**", and submit your review.

Step A.9: Get Merged

After your Lecturer and classmates have reviewed your PR, the Lecturer will "**Merge**" it. Your code is now officially part of the original project. Congratulations!

IV. Part B: Group Project Setup (Main Task)

Now, you will apply this process to set up your group's repository for the course project. This time, the workflow is slightly different because you are all "*Collaborators*" on one central repo.

Step B.1: Form Groups

- Form your groups of **2-3 students**.
- Designate **one person** as the "*Team Leader*". This person will create and manage the group's main repository.

Step B.2: (Team Leader) Create Group Repo

1. The lecturer provides a "*Base Project*" as starter pack for group project `https://github.com/longndt/devops-project`
2. **Team Leader Only: Fork** this "*Base Project*" to your account.
3. Go to the newly forked repo on your (the *Team Leader*'s) account. Go to `Settings`:
    - **Rename** the repository to your group's name (e.g., `devops-group-1`).

Step B.3: (Team Leader) Add Collaborators

1. In your group's new repo (`devops-group-1`), go to `Settings` > `Collaborators`.
2. Add **all your group members** (by their GitHub usernames) and your **Lecturer** as collaborators.
3. Give them **"Write"** or **"Maintain"** access. Your team members must accept the invitation via email.

Step B.4: (All Members) Clone the Group Repo

All members (including the *Team Leader*) will now `clone` the new group repo (**NOT** their own fork) to their local machines.

```
# Everyone runs this command, using the Team Leader's URL
$ git clone https://github.com/LeaderUsername/devops-group-1.git
```

```
$ cd devops-group-1
```

Step B.5: Team Workflow Test (The "Collaborator" Workflow)

Verification Task: To ensure everything is set up correctly, **every member of the group** must practice the workflow on your new group repo.

1. **Each Member:** Create a new branch (e.g., `feature/add-john`).

```
$ git checkout -b feature/add-your-name
```

2. **Each Member:** Edit the `README.md` file to add your name.
3. **Each Member:** Commit and Push your branch. Because you are a collaborator, you can push directly to the group repo.

```
$ git add README.md

$ git commit -m "docs: Add [Your Name] to README"

$ git push origin feature/add-your-name
```

4. **Each Member:** Open a Pull Request. This time, the **base and head repositories are the same** (your group's repo).
5. **Other Members:** You MUST review and "**Approve**" your teammates' PRs.
6. **Team Leader:** You are responsible for "**Merging**" the approved PRs.

VI. Common Issues & Troubleshooting

- **Error: "Permission denied (publickey)."**

  Fix: You are trying to `push` to a repo you don't have write access to. Make sure

you are pushing to `origin` (your fork) in Part A, or that the Team Leader has added you as a Collaborator in Part B.

- **Error: "Updates were rejected because the tip of your current branch is behind..."**
  Fix: Someone else pushed to the `main` branch. You must pull their changes first. Run `git pull origin main` before pushing.
- **Merge Conflicts:** This happens if two people edit the same line in the same file.
  Fix: GitHub will show the conflict. The easiest fix is to run `git pull origin main` on your branch, fix the file locally (VS Code shows conflict markers), `git add .`, `git commit`, and `git push` again.